FAST - NU

# High Performance Computing with GPUs

**Deliverable 04**


Mohammad Faran Azam (23i-0075) _____
Saim Ahmad (23i-0781) _____
Akif Jawad (23i-0583) _____
11-17-2025

## Abstract

The Kanade–Lucas–Tomasi (KLT) feature tracker is a core component of real-time vision systems, yet its computationally intensive gradient and convolution operations limit performance on high-resolution or embedded platforms. Our analysis of a legacy CUDA-based implementation revealed bottlenecks in functions such as ComputeGradientSum, computeIntensityDifference, and the convolution modules. These inefficiencies stemmed from heavy manual memory management, uneven parallelism, and repeated host–device transfers.

To address these issues, we redesigned the pipeline using OpenACC, a directive-based GPU programming model offering portability and reduced development overhead. Key optimizations included minimizing explicit memory handling through unified memory, restructuring convolution loops for improved data locality, offloading compute-dense regions using OpenACC parallel constructs, and incorporating asynchronous execution to exploit device concurrency. The resulting OpenACC implementation significantly improves throughput while maintaining algorithmic correctness and simplifying the codebase.

Our findings demonstrate that directive-based acceleration can provide performance close to handcrafted CUDA, offering a practical and maintainable pathway for modernizing legacy vision algorithms with minimal code disruption.

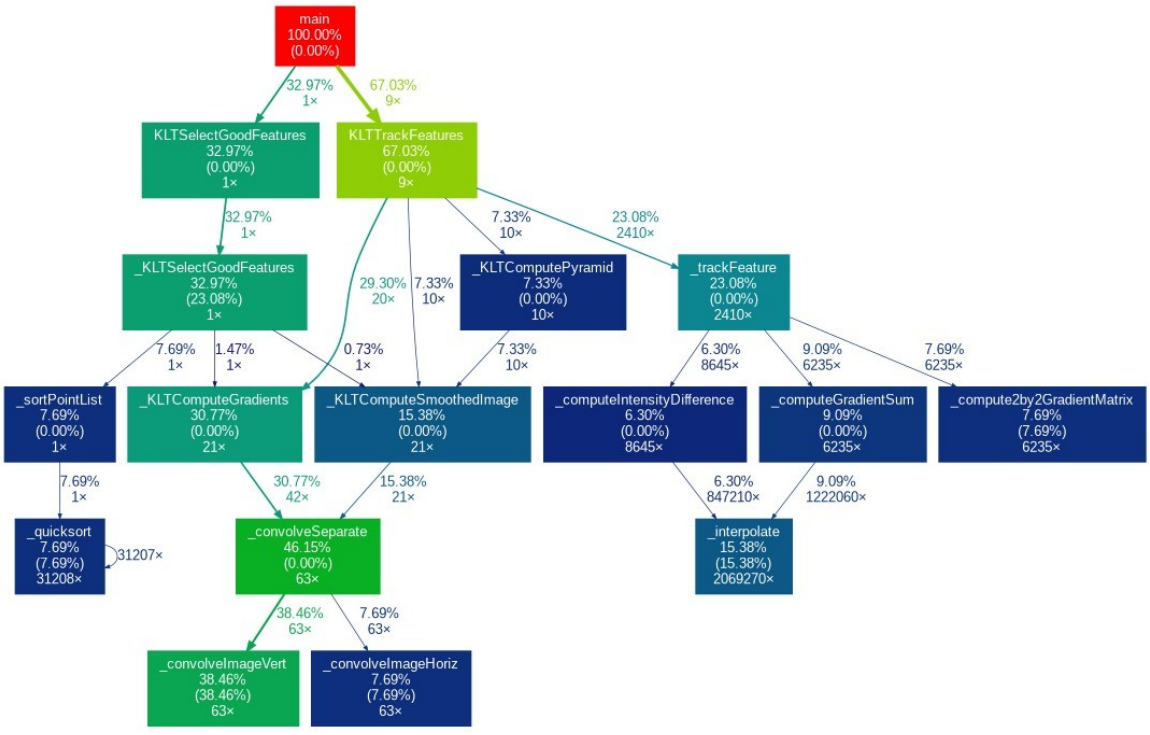
## Introduction

The original KLT implementation was written for CPU execution, resulting in substantial runtime overhead due to deeply nested loops in gradient and convolution routines. To improve efficiency, we adopted OpenACC rather than CUDA-specific kernels, enabling GPU acceleration with minimal changes to the underlying code. By annotating critical loops with #pragma acc directives—including parallel and loop constructs—we were able to migrate high-cost computations to the GPU while maintaining code clarity and portability.

## Methodology

The optimization process focused on profiling, identifying computational hotspots, and selectively offloading them to the GPU with OpenACC.

Profiling showed that the horizontal, vertical, and separable convolution functions dominated execution time due to their nested per-pixel computations.




These functions were refactored for GPU execution while preserving their structure. **Gaussian smoothing** and derivative kernels were first computed on the host, then transferred once to the device using persistent enter **data regions**. Updates were handled through **update device** to avoid redundant transfers.

Convolution loops were offloaded using explicit OpenACC parallelism. The outer loop mapped to **gang parallelism**, while the inner column loop used **vector parallelism** for coalesced memory access. The innermost kernel-weight loop remained sequential to ensure correctness. Border handling was separated into dedicated loops to minimize control divergence on the GPU.

Intermediate results in separable convolution were allocated directly on the device using **acc_malloc** and managed through enter data and exit data directives, eliminating intermediate host–device transfers. Higher-level functions such as _KLTComputeGradients and _KLTComputeSmoothedImage were enclosed in data regions to keep all arrays resident on the GPU throughout execution.

This methodology provided high-performance GPU acceleration while preserving portability and avoiding the complexity of explicit CUDA kernels.

- #pragma acc data – Defines a region where arrays are copied to/from the device once, avoiding repeated transfers.
- #pragma acc parallel – Launch a GPU kernel region; compiler may schedule gangs/vectors automatically.
- #pragma acc loop gang – Assign loop iterations to “gangs” (CUDA thread blocks / coarse parallelism).
- #pragma acc loop vector – Assign loop iterations to vector lanes (threads within a block).
- #pragma acc loop seq – Force the loop to be executed sequentially (single thread), usually for small inner loops.
- #pragma acc enter data – Persistently allocate and copy host data to the GPU outside a compute region.
- #pragma acc exit data – Free GPU memory allocated by enter data.
- #pragma acc update device(...) – Copy host → device for data that already has a GPU allocation.

## CUDA Implementation vs. OpenACC Implementation

| Aspect | OpenACC Implementation | Direct CUDA Implementation |
|---|---|---|
| **Development Effort** | Low; add directives to existing loops with minimal | High; requires custom kernels, thread indexing, and |

| Aspect | OpenACC Implementation | Direct CUDA Implementation |
|---|---|---|
| | restructuring. | synchronization. |
| **Memory Management** | Unified memory and implicit transfers; minimal manual handling. | Fully manual; cudaMalloc, cudaMemcpy, and lifetime management required. |
| **Parallelism Control** | Abstracted; compiler decides gang/vector layout. | Fine-grained control over threads, blocks, shared memory, and warps. |
| **Performance** | High for regular loop-based workloads; near-CUDA for convolutions. | Potentially highest with shared-memory and warp-level optimizations. |
| **Maintainability & Portability** | Very high; portable across OpenACC-capable systems. | Lower; tied to NVIDIA GPUs and hardware-specific tuning. |

For the KLT workload, OpenACC offered speedups with lower development effort compared to custom CUDA version, which could achieve additional micro-optimizations

## Results

The code was run on image (3840x2160) sequence of 5000 features at 100 frames. The execution times recorded are as follows:

**CPU execution time:**

```
(KLT) Tracking 528 features in a 3840 by 2160 image...
        523 features successfully tracked.
(KLT) Tracking 523 features in a 3840 by 2160 image...
        519 features successfully tracked.
(KLT) Writing feature table to text file: '/home/23I-0583/klt/sample/features.txt'
(KLT) Writing feature table to binary file: '/home/23I-0583/klt/sample/features.ft'
total time by track feature: 47737.581 ms
23I-0583@HPC02:~/PROJECTv1/klt$
```

**GPU execution time:**

```
(KLT) Tracking 528 features in a 3840 by 2160 image...
        523 features successfully tracked.
(KLT) Tracking 523 features in a 3840 by 2160 image...
        519 features successfully tracked.
(KLT) Writing feature table to text file: '/home/23I-0583/klt/sample/features.txt'
(KLT) Writing feature table to binary file: '/home/23I-0583/klt/sample/features.ft'
total time by track feature: 36165.335 ms
23I-0583@HPC02:~/PROJECTv1/klt$
```

**Speed up** = 47737 / 36165

= 1.32x

**Key Benefits**

- Major throughput improvement, enabling near real-time performance.
- Considerably reduced code complexity compared to writing custom CUDA kernels.
- Elimination of manual data transfers improved reliability and maintainability.
- Improved scalability for higher resolutions and feature counts.

**Limitations**

- OpenACC abstraction restricts fine-grained optimizations achievable with CUDA shared memory.
- Asynchronous execution and prefetching yielded only moderate gains in some scenarios.